

LISTA DE EXERCÍCIOS

Seleção de exercícios, alguns da referência [1] no Moodle.

1 Análise Assintótica

Exercício 1. Descreva um algoritmo para computar o máximo valor em uma sequência $\langle a_1, a_2, \dots, a_n \rangle$. Mostre, ao lado de cada linha do seu algoritmo, o número exato de vezes que a linha é executada e a aproximação assintótica. Por fim, descreva a ordem de complexidade assintótica do seu algoritmo.

Exercício 2. Descreva um algoritmo para encontrar um valor x em uma sequência $\langle a_1, a_2, \dots, a_n \rangle$ ordenada. Mostre, ao lado de cada linha do seu algoritmo, o número exato de vezes que a linha é executada e a aproximação assintótica. Por fim, descreva a ordem de complexidade assintótica do seu algoritmo.

Exercício 3. Considere um vetor com $2n$ elementos de forma que a primeira e a segunda metades formam sequências ordenadas (cada uma com n elementos). Veja um exemplo abaixo.

1	2	3	4	5	6	7	8
2	3	5	8	1	4	6	7

Descreva um algoritmo que gere como saída um vetor que contém a sequência de $2n$ elementos ordenada. Para o exemplo anterior, teríamos o vetor abaixo.

1	2	3	4	5	6	7	8
1	2	3	4	5	6	7	8

Exercício 4. Sejam n pontos no plano, $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, descreva um algoritmo para computar os dois pontos mais próximos conforme a distância euclidiana. Em seguida, analise a complexidade do seu algoritmo.

Exercício 5. Seja $f(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n^1 + a_0$ um polinômio de grau k com coeficiente a_k positivo. Mostre que $f(n) = O(n^k)$.

Exercício 6. Mostre que $\log_a n = \Theta(\log_b n)$ para a variável $n > 1$ e constantes $a > 1$ e $b > 1$.

Exercício 7. Descreva um algoritmo para computar o máximo valor em uma sequência $\langle a_1, a_2, \dots, a_n \rangle$. Prove a corretude do seu algoritmo usando um loop invariante. Em seguida, analise a complexidade do seu algoritmo no melhor caso, no pior caso e no caso médio.

Exercício 8. Indique se $f(n) \in O(g(n))$ para cada par de funções abaixo:

a) $f(n) = n(n+1)$ e $g(n) = n^2$;

b) $f(n) = 333n^3$ e $g(n) = n^2$;

c) $f(n) = \lg n$ e $g(n) = \log n$;

d) $f(n) = 4^n$ e $g(n) = 2^n$.

Exercício 9. Descreva a função $T(n)$ que caracteriza o número de execuções de cada linha do algoritmo abaixo.

```
fun(A[1..n,1..n])  
  para i de 1 até n-1  
    para j de i+1 até n  
      se A[i, j] diferente de A[j, i]  
        retorna falso  
  retorna verdadeiro
```

Exercício 10. Mostre que $\sum_{i=1}^n i = 1 + 2 + \dots + n = \frac{n(n+1)}{2}$.

Exercício 11. Mostre que $\sum_{i=0}^n a^i = a^0 + a^1 + \dots + a^n = \frac{a^{n+1} - 1}{a - 1}$.

2 Divisão e Conquista

Exercício 12. Implemente os algoritmos Insertion Sort e Merge Sort. Usando alguma ferramenta de sua preferência, produza um gráfico comparando os tempos dos dois algoritmos para tamanhos de entrada diferentes. Para quais tamanhos de entrada o Insertion Sort é mais rápido e para quais tamanhos de entrada o Merge Sort é mais rápido.

Exercício 13. Usando o método de substituição, prove os limites abaixo:

a) $T(n) = T(n-1) + n$ é $O(n^2)$

b) $T(n) = T(n/2) + 1$ é $O(\log n)$

c) $T(n) = 2T(n/2) + n^2$ é $O(n^2)$

d) $T(n) = 3T(n/4) + n^2$ é $O(n^2)$

Exercício 14. Considere uma sequência de números distintos $A = \langle a_1, a_2, \dots, a_n \rangle$. Dois índices $i < j$ formam uma inversão se $a_i > a_j$, ou seja, os dois elementos estão fora de ordem. Descreva um algoritmo que determine o número de inversões em uma sequência A .

Exercício 15. Considere dois conjuntos de números (sem números iguais) $\{a_1, a_2, \dots, a_n\}$ e $\{b_1, b_2, \dots, b_n\}$. Assuma que a única forma de acessar qualquer valor dos conjuntos é pela função `menor(S, k)`, que retorna o k -ésimo menor valor do conjunto S . Por exemplo, `menor(A, 10)` retorna o décimo menor valor dentre os n valores do conjunto A . Considerando os dois conjuntos de números, a mediana será o n -ésimo menor valor dentre os $2n$ existentes. Descreva um algoritmo para encontrar a mediana dos $2n$ valores usando $O(\log n)$ chamadas à função `menor(S, k)`.

Exercício 16. Descreva algoritmos para criar e manipular filas de prioridades construídas usando heaps de máximo. Sua fila de prioridades deve ser capaz de fazer as seguintes operações:

- criar uma fila de prioridade (heap de máximo) dados n números inteiros;
- retornar o maior valor na fila de prioridade;
- remover o maior valor da fila de prioridade;
- inserir um novo valor na fila de prioridade;
- aumentar o valor de um elemento na fila de prioridade.

Em seguida, analise a ordem de complexidade de tempo para cada algoritmo.

Exercício 17. Descreva um algoritmo de divisão e conquista para computar o número de bits necessários para representar um valor natural n e, em seguida, analise a complexidade do seu algoritmo.

Exercício 18. Considerando o algoritmo abaixo, que recebe um inteiro positivo n :

```

S3(n)
  se n = 1
    retorna 1
  senão
    retorna S(n-1)+n*n*n

```

- descreva o que o algoritmo computa;
- mostre a relação de recorrência para o algoritmo;
- resolva a relação usando o método da árvore de recursão.

Exercício 19. Escreva um algoritmo de divisão e conquista para encontrar a posição do maior elemento em um array de n números. Em seguida mostre a relação de recorrência para o seu algoritmo e resolva a relação usando o método da árvore de recursão.

Exercício 20. Proponha um algoritmo $O(\lg n)$ de divisão e conquista para computar o valor da função $f(n) = 2^n$. Mostre a relação de recorrência e, usando o método da árvore de recursão, que seu algoritmo cumpre a ordem de complexidade exigida.

Exercício 21. Usando o método da árvore de recursão, mostre que:

- $T(n) = 9T(n/3) + n \in O(n^2)$
- $T(n) = 9T(n/3) + n^2 \in O(n^2 \log_3 n)$
- $T(n) = 9T(n/3) + n^3 \in O(n^3)$

Faça o mesmo para as relações de recorrência no Exercício 13.

Exercício 22. Resolva as relações de recorrência no Exercício 21 usando o Teorema Mestre.

Exercício 23. Dados um conjunto de n números inteiros e um valor inteiro x , proponha um algoritmo de tempo $O(n \lg n)$ que verifique se há dois números no conjunto cuja soma é igual a x .

3 Grafos

Nas questões seguintes, assuma que um grafo é dado por uma lista de adjacências sempre que não for mencionada a representação.

Exercício 24. Dado um grafo direcionado representado por uma lista de adjacências, descreva como é possível computar o grau de entrada (número de arestas que chegam em um vértice) e o grau de saída (número de arestas que saem de um vértice) para todos os vértices em tempo $O(n + m)$.

Exercício 25. Prove que em todo grafo direcionado e sem ciclos sempre existe um vértice com grau de entrada (número de arestas que chegam no vértice) 0.

Exercício 26. Um vértice sorvedouro em um grafo direcionado é um vértice cujo grau de saída é zero. Descreva um algoritmo que recebe um grafo direcionado como entrada e retorna um vértice sorvedouro se ele existir. Analise a complexidade assintótica do seu algoritmo.

Exercício 27. Dado um grafo direcionado $G = (V, E)$, seu grafo transposto é dado por $G^T = (V, E^T)$, com $E^T = \{(u, v) \in V \times V : (v, u) \in E\}$. Descreva algoritmos para computar o grafo transposto se:

- G e G^T são representados por listas de adjacências;
- G e G^T são representados por matrizes de adjacências.

Exercício 28. Qual seria a complexidade do algoritmo de busca em largura visto em aula caso o grafo fosse representado por uma matriz de adjacências?

Exercício 29. Em buscas em grafos, as arestas que levam os vértices a serem visitados formam uma árvore de busca. Modifique o algoritmo de busca em largura visto em aula para que ele armazene um campo “predecessor” em cada vértice. O predecessor $v.p$ de um vértice v do grafo deve ser *nulo* se v é a raiz da árvore, ou um vértice u se v foi visitado a partir de uma aresta (u, v) . Após modificar o algoritmo de busca, escreva um algoritmo que, dado um vértice v do grafo, imprime todos os vértices da árvore entre a raiz e o vértice v .

Exercício 30. Argumente que a árvore de busca de uma busca em largura pode não ser única.

Exercício 31. Durante uma busca em profundidade, as arestas de um grafo podem ser classificadas em:

- aresta de árvore: (u, v) tal que v foi descoberto ao explorar a aresta (u, v) ;
- aresta de retorno: (u, v) conecta u com um ancestral v na árvore de busca em profundidade;
- aresta de avanço: (u, v) conecta u com um descendente v na árvore de busca em profundidade, mas não é uma aresta de árvore;
- aresta de cruzamento: as outras arestas, podem ocorrer em uma mesma árvore de busca em profundidade, desde que um vértice não seja ancestral do outro, ou em árvores diferentes.

Modifique o algoritmo de busca em profundidade para classificar as aresta de um grafo direcionado.

Exercício 32. Dado um grafo $G = (V, E)$ e um vértice $s \in V$, escreva um algoritmo para computar o caminho mínimo entre s e todo $v \in V$. A distância em um caminho $s \rightsquigarrow v$ é definida como o número de arestas no caminho. Assuma distância infinita para vértices que não podem ser alcançados a partir de s .

Exercício 33. Usando a saída do seu algoritmo para o Exercício 32, escreva um algoritmo que recebe um vértice v e imprime a sequência de vértices que formam o caminho mínimo $s \rightsquigarrow v$, se este existir.

Exercício 34. Escreva um algoritmo para detectar a existência de ciclos em grafos não direcionados.

Exercício 35. Seja T uma árvore de busca em largura sobre um grafo G e x e y dois nós de T nas camadas i e j , respectivamente. Considerando que (x, y) é uma aresta de G , prove que a diferença entre os níveis i e j é no máximo 1.

4 Algoritmos Gulosos

Exercício 36. Mostre que a escolha gulosa da atividade que começa mais tarde gera uma solução ótima para o problema de seleção de atividades.

Exercício 37. Suponha um conjunto de n itens em que cada item possui um peso positivo p_i disponível e um valor positivo v_i , relacionado com o peso disponível. Você precisa colocar itens em uma mochila que consegue carregar um peso de até W . Para isso, deve escolher uma fração $x_i \leq p_i$ de cada item, de forma que a capacidade da mochila não é excedida e o valor da mochila é maximizado. Descreva um algoritmo guloso para este problema e prove que ele gera uma solução ótima.

Exercício 38. Considere novamente o problema da mochila do exercício anterior. No entanto, desta vez um item não pode ser parcialmente colocado na mochila, ou seja, ao escolher um item i , este ocupará um peso w_i na mochila (se possível). Considere também, que a sequência de itens ordenados por peso crescente é igual à sequência de itens ordenados por valor decrescente. Descreva um algoritmo guloso que maximize o valor da mochila e prove que ele gera uma solução ótima.

Exercício 39. Você e seus colegas farão uma caminhada de alguns dias entre duas cidades. Vocês sabem que conseguem caminhar, no máximo, d quilômetros por dia e que existem pontos para acampar localizados nas distâncias $p_1, p_2, \dots, p_n$ a partir do início da caminhada. A distância entre dois pontos de parada subsequentes não é maior que d . Proponha um algoritmo guloso para encontrar o menor número de paradas e que permita que vocês cheguem ao destino. Mostre que seu algoritmo funciona.

Exercício 40. Suponha um arquivo de 1.000.000 de caracteres formado pelos alfabeto a, b, c e d , com frequências $1/2, 1/4, 1/8, 1/16$ e $1/16$, respectivamente.

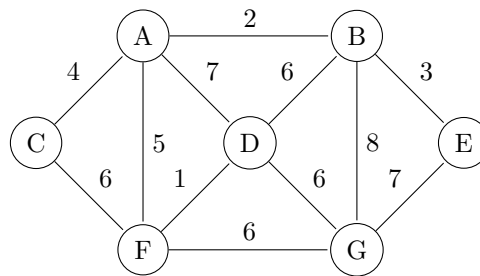
- Construa a árvore para o código de Huffman deste alfabeto.
- Qual é o código binário para cada símbolo?
- Qual é o tamanho do arquivo, em bits, usando o código de Huffman?

Exercício 41. Argumente que uma árvore de codificação ótima é uma árvore completa.

Exercício 42. Generalize o algoritmo de Huffman para códigos ternários, que usam os símbolos 0, 1 e 2. Argumente que o seu algoritmo gera codificações ótimas.

Exercício 43. Dado um $G = (V, E)$, grafo não direcionado e sem pesos, e dois vértices $u, v \in V$, dê um algoritmo que compute o número de caminhos mínimos distintos entre u e v .

Exercício 44. Execute o algoritmo de Dijkstra sobre o grafo abaixo, partindo do vértice A .

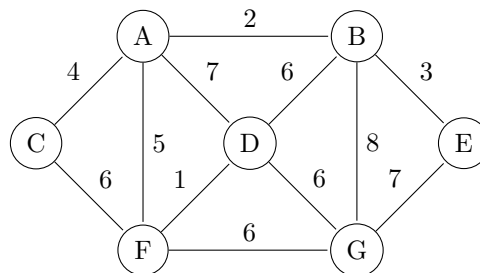


- Mostre uma tabela com as distâncias intermediárias para cada nó em cada iteração.
- Mostre a árvore de distâncias final.

Exercício 45. Um caminho mínimo pode conter ciclos? Por quê?

Exercício 46. Seja $G = (V, E)$ um grafo direcionado, acíclico e com pesos. Dê um algoritmo linear, no tamanho do grafo, para computar os caminhos mínimos de $s \in V$ para todo $v \in V$. Seu algoritmo funciona se o grafo possuir arestas com pesos negativos? Por quê?

Exercício 47. Execute os algoritmos de Prim e Kruskal sobre o grafo abaixo.



- Mostre uma tabela, para cada algoritmo, que contenha as escolhas importantes em cada iteração.
- Mostre a AGM final.

Exercício 48. Se (u, v) uma aresta com menor peso em um grafo G , mostre que ela pertence a alguma AGM.

Exercício 49. Mostre que um grafo possui uma única AGM se para todo corte existe uma única aresta leve (aresta com menor peso). Usando um contra-exemplo, mostre que o contrário não é verdade.

Exercício 50. Modifique o algoritmo de Prim para usar uma matriz de adjacências. Descreva a complexidade de tempo do novo algoritmo.

5 Programação dinâmica

Exercício 51. O que significa o termo memoização?

Exercício 52. Compute a matriz para a Maior Subsequência Comum (MSC) dadas as cadeias de entrada $X = actgctac$ e $Y = catcgta$.

Exercício 53. Descreva a ordem de complexidade do algoritmo recursivo que imprime uma MSC.

Exercício 54. Exercício 15-1 (Cormen terceira edição).

Exercício 55. Seja d a matriz tridimensional resultante da saída do algoritmo Floyd-Warshall, mostre que ciclos negativos podem ser detectados consultando as células $d[i, i, n]$, sendo $1 \leq i \leq n$ e n o número de vértices.

Exercício 56. Dadas duas sequências de caracteres X e Y , podemos determinar operações para transformar X em Y . O problema pode ser visto como o alinhamento entre os caracteres das duas sequências. Abaixo temos dois exemplos de alinhamentos para as sequências $X = \text{SNOWY}$ e $Y = \text{SUNNY}$.

S		N	O	W	Y		S	N	O	W		Y	
S	U	N	N		Y		S	U	N			N	Y

O custo de um alinhamento é o número de colunas em que os caracteres diferem. No exemplo acima, o primeiro alinhamento tem custo 3 e o segundo tem custo 5. A Distância de Edição é igual ao menor custo de alinhamento e é baseada nas seguintes operações:

- substituição: troca um caractere de X por outro;
- inserção: insere um caractere em X ;
- remoção: remove um caractere de X .

No primeiro alinhamento do exemplo, temos três operações de edição: inserção de “U”, substituição de “O” por “N”, e remoção de “W”.

Considerando que a entrada para o problema da Distância de Edição são duas sequências de caracteres dadas pelos vetores $X[1 \dots n]$ e $Y[1 \dots m]$. Resolva o problema usando programação dinâmica e mostre:

- quais são os subproblemas que precisam ser resolvidos para resolver o problema original;
- quantos são esses subproblemas;
- quais são as escolhas em cada passo e quais são os subproblemas que surgem;
- a prova de que o problema contém subestrutura ótima;
- o algoritmo;

- f) a complexidade de tempo do algoritmo;
- g) a aplicação do algoritmo sobre as sequências $X = ALGORITHM$ e $Y = ALTRUISTIC$.

Exercício 57. Considere uma sequência de n moedas com valores inteiros positivos $c_1, c_2, \dots, c_n$. Proponha uma solução, usando programação dinâmica, para recolher o máximo valor escolhendo moedas na sequência dada, sob a restrição de não poder escolher moedas adjacentes na sequência.

6 Fluxo Máximo

Exercício 58. Suponha um grafo direcionado com pesos, $G = (V, E)$, que possui múltiplas origens $\{s_1, s_2, \dots, s_k\} \in V$ e sorvedores $\{t_1, t_2, \dots, t_l\} \in V$. Mostre como reduzir o problema de computar o fluxo máximo em G para o problema de computar o fluxo máximo em um grafo com uma única origem s e um único sorvedor t (algoritmo de Ford-Fulkerson).

Exercício 59. Dado um fluxo em rede $G = (V, E)$, com arestas de capacidade unitária, e um inteiro positivo k , proponha uma maneira de remover k arestas de G de forma a obter a maior redução possível do fluxo máximo. Por que sua estratégia funciona?

Exercício 60. Considere uma rede de fluxos $G = (V, E)$ com capacidades inteiras positivas c_e em cada aresta $e \in E$ e os valores f_e para um fluxo máximo em G . Descreva um algoritmo $O(n + m)$ que determina se é possível aumentar o tamanho do fluxo se a capacidade de uma aresta de G é aumentada em uma unidade.

Exercício 61. Considere que no grafo residual do algoritmo de Bellman-Ford não seja permitido criar arestas chegando em s . Neste caso, argumente que o algoritmo continua funcionando.

Exercício 62. Um (s, t) -corte em uma rede de fluxos é único? E se todas as capacidades forem distintas?